



Politechnika Wrocławska

Sprawozdanie: Projekt 2

Kolejki priorytetowe

Grupa 4 | Prowadzący: mgr inż. Piotr Nowak

https://github.com/melamann00/proj2_struktury_danych.git

Jan Nowak

284138

Filip Bassa

284183

1 Cel projektu

Celem projektu było zaimplementowanie kolejki priorytetowej w języku C++ z użyciem trzech różnych struktur danych oraz porównanie wydajności następujących operacji:

- **push** – dodawanie elementu
- **pop** – usuwanie elementu o najwyższym priorytecie
- **peek** – podejrzenie elementu o najwyższym priorytecie bez usuwania
- **size** – zwrócenie rozmiaru kolejki
- **changePriority** – modyfikacja priorytetu wskazanego elementu

Porównanie przeprowadzono dla trzech rozmiarów danych ($N = 1\,000, 10\,000, 100\,000$) oraz trzech przypadków: optymistycznego, średniego i pesymistycznego.

2 Opis implementacji

2.1 BinaryHeapPQ – kopiec binarny

Kolejka zrealizowana na tablicy dynamicznej w postaci kopca binarnego. Węzeł przechowuje wartość i priorytet. Kopiec spełnia warunek: każdy rodzic ma priorytet większy lub równy potomkom.

- **push**: wstawienie na koniec, **siftUp** – $O(\log n)$
- **pop**: zamiana korzenia z ostatnim elementem, **siftDown** – $O(\log n)$
- **peek**: bezpośredni dostęp do **h[0]** – $O(1)$
- **size**: rozmiar wektora – $O(1)$
- **changePriority**: liniowe wyszukanie + **siftUp/siftDown** – $O(n + \log n)$

Przypadek optymistyczny dla **push**: dane posortowane rosnąco. Przypadek pesymistyczny: dane posortowane malejąco.

2.2 SortedArrayPQ – posortowana tablica

Kolejka przechowuje elementy w tablicy posortowanej rosnąco według priorytetu. Element o najwyższym priorytecie zawsze znajduje się na końcu tablicy.

- **push**: wyszukiwanie binarne pozycji i wstawienie – $O(n)$
- **pop**: usunięcie ostatniego elementu – $O(1)$
- **peek**: dostęp do ostatniego elementu – $O(1)$
- **changePriority**: liniowe wyszukanie, usunięcie i ponowne wstawienie – $O(n)$

Przypadek optymistyczny dla **push**: dane posortowane rosnąco – nowy element jest zawsze wstawiany na koniec (brak przesunięcia). Przypadek pesymistyczny: dane malejąco – wstawienie na początek = przesuwanie całej tablicy.

2.3 SortedListPQ – posortowana lista dwukierunkowa

Kolejka zrealizowana na `std::list`, utrzymywana w porządku malejącym priorytetu. Element o najwyższym priorytecie jest zawsze na początku listy.

- **push**: liniowe wyszukanie pozycji i wstawienie (bez kopiowania) – $O(n)$
- **pop**: usunięcie pierwszego elementu – $O(1)$
- **peek**: dostęp do pierwszego elementu – $O(1)$
- **changePriority**: liniowe wyszukanie, usunięcie i ponowne wstawienie – $O(n)$

Przypadek optymistyczny dla **push**: dane posortowane malejąco – każdy nowy element trafia od razu na koniec listy. Przypadek pesymistyczny: dane rosnąco – nowy element jest zawsze wyszukiwany aż do końca listy.

3 Wyniki pomiarów

Czasy **push**, **pop** i **changePriority** podano w mikrosekundach [μs] dla całej sekwencji N operacji. **Peek** i **size** mierzono jako średnią z 200 powtórzeń [ns].

Struktura	Przypadek	N	push [μ s]	pop [μ s]	peek [ns]	size [ns]	chgPri [μ s]
BinaryHeap	optymistyczny	1 000	198	238	23	12	0
BinaryHeap	średni	1 000	66	271	21	11	0
BinaryHeap	pesymistyczny	1 000	35	257	24	11	0
SortedArray	optymistyczny	1 000	97	16	31	12	2
SortedArray	średni	1 000	186	15	31	9	2
SortedArray	pesymistyczny	1 000	153	16	24	4	2
SortedList	optymistyczny	1 000	5 683	36	38	12	1
SortedList	średni	1 000	2 916	47	35	11	1
SortedList	pesymistyczny	1 000	76	34	37	14	0
BinaryHeap	optymistyczny	10 000	2 337	3 345	23	12	0
BinaryHeap	średni	10 000	704	3 667	21	11	3
BinaryHeap	pesymistyczny	10 000	424	3 421	21	11	2
SortedArray	optymistyczny	10 000	1 296	159	31	9	24
SortedArray	średni	10 000	4 206	154	31	9	25
SortedArray	pesymistyczny	10 000	7 028	159	36	11	24
SortedList	optymistyczny	10 000	556 747	343	35	14	6
SortedList	średni	10 000	280 749	348	35	11	7
SortedList	pesymistyczny	10 000	804	342	35	12	5
BinaryHeap	optymistyczny	100 000	28 571	42 924	16	11	12
BinaryHeap	średni	100 000	7 681	46 761	23	11	15
BinaryHeap	pesymistyczny	100 000	4 118	43 673	21	7	24
SortedArray	optymistyczny	100 000	17 001	1 538	46	9	270
SortedArray	średni	100 000	320 551	1 549	28	12	253
SortedArray	pesymistyczny	100 000	660 475	1 556	27	9	246
SortedList	optymistyczny	100 000	57 027 042	3 480	33	11	57
SortedList	średni	100 000	51 975 453	5 016	38	14	191
SortedList	pesymistyczny	100 000	10 999	5 562	36	11	122

4 Analiza wyników

4.1 Operacja push

BinaryHeap ma stabilną złożoność $O(\log n)$ we wszystkich przypadkach. Dla $N = 100\,000$ czas wzrasta tylko 15 razy wraz z 100 - krotnym wzrostem N , co potwierdza złożoność logarytmiczną. SortedArray jest bardzo szybki dla przypadku optymistycznego, ale bardzo spowalnia dla przypadku pesymistycznego – $660\,475\ \mu s$ dla $N = 100\,000$ vs. $17\,001\ \mu s$ dla optymistycznego. SortedList jest najgorsza dla przypadków średniego i optymistycznego z powodu braku lokalności pamięci i liniowego przeszukiwania. Dla $N = 100\,000$ i przypadku średniego aż ok. 52 sekundy.

4.2 Operacja pop

Złożoność obliczeniowa SortedArray i SortedList dla `pop` wynosi $O(1)$ – usuwają z końca/początku bez przebudowy. BinaryHeap wykonuje `siftDown` po każdym `pop`, co daje $O(\log n)$. Czas SortedArray `pop` to ok. $1\,538\text{--}1\,556\ \mu s$, dla $N = 100\,000$ BinaryHeap to ok. $42\,924\text{--}46\,761\ \mu s$.

4.3 Operacja changePriority

Wszystkie trzy struktury wymagają liniowego wyszukania elementu. BinaryHeap wykonuje dodatkowo `siftUp/siftDown`, co teoretycznie daje $O(n + \log n)$, lecz w praktyce czas jest porównywalny z SortedArray. SortedList dla przypadku średniego przy $N = 100\,000$ wypada najgorzej ($191\ \mu s$) ze względu na brak lokalności pamięci przy wyszukiwaniu.

4.4 Peek i size

Obie operacje są $O(1)$ dla wszystkich struktur i wykonują się w czasie kilkunastu nanosekund, co potwierdza stały czas dostępu do pierwszego i ostatniego elementu i pola rozmiaru.

5 Złożoność obliczeniowa

Operacja	BinaryHeap	SortedArray	SortedList
push	$O(\log n)$	$O(n)$	$O(n)$
pop	$O(\log n)$	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$	$O(1)$
changePriority	$O(n + \log n)$	$O(n)$	$O(n)$

Tabela 2: Złożoność obliczeniowa operacji dla każdej struktury. `changePriority` w BinaryHeap: $O(n)$ na wyszukanie + $O(\log n)$ na `siftUp/siftDown` = $O(n)$ łącznie.

6 Wnioski

Na podstawie przeprowadzonych pomiarów i analizy złożoności można sformułować następujące wnioski:

- **BinaryHeap** jest najlepiej zbilansowaną strukturą. Zapewnia $O(\log n)$ dla `push` i `pop`, co czyni go odpowiednim wyborem gdy obie operacje są wykonywane często. Wydajność jest przewidywalna niezależnie od początkowego porządku danych.
- **SortedArray** sprawdza się, gdy `push` jest rzadki lub dane są wstawiane w porządku rosnącym, natomiast `pop` jest częsty i szybki. W przypadku pesymistycznym staje się jednak praktycznie bezużyteczny dla dużych N ze względu na masywne przesuwanie elementów tablicy.
- **SortedList** oferuje $O(1)$ `pop` i `peek`, jednak jest najgorsza z trzech struktur pod kątem `push` w przypadkach średnich i optymistycznych, co wynika z braku lokalności pamięci i konieczności iterowania po liście. Nadaje się jedynie jeśli dane są wstawiane w porządku malejącym priorytetu.
- Dla zastosowań produkcyjnych z mieszanym obciążeniem zdecydowanie zalecane jest BinaryHeap lub – w przypadku zaawansowanych wymagań – kopiec Fibonacciego ($O(1)$ amortyzowane dla `push`, $O(\log n)$ dla `pop`).